

1. Find employees whose salary is greater than the department average, but only in departments with more than N people.

Explanation: This query requires two levels of filtering. First, we use a subquery to calculate the average salary and employee count per department. Second, we filter that subquery to include only departments where the count is greater than N. Finally, we join the result back to the Employees table to find individuals whose salary exceeds their filtered department's average.

Query (N=10):

```
SELECT e.employee_id, e.name, e.salary, d.dept_avg
FROM Employees e
JOIN (
    SELECT
        department_id,
        AVG(salary) AS dept_avg,
        COUNT(employee_id) AS dept_count
    FROM Employees
    GROUP BY department_id
    HAVING dept_count > 10 -- Filter 1: Departments with > N people
) AS d ON e.department_id = d.department_id
WHERE e.salary > d.dept_avg; -- Filter 2: Employee salary > dept average
```

2. Find customers who bought every product in a given category.

Explanation: This is a classic "Division" problem. We need to find customers where the count of distinct products they ordered in the category equals the total count of products available in that category.

Query:

```
SELECT customer_id
FROM Orders o
JOIN OrderDetails od ON o.order_id = od.order_id
JOIN Products p ON od.product_id = p.product_id
WHERE p.category_name = 'Electronics' -- Specify the category
GROUP BY customer_id
HAVING COUNT(DISTINCT p.product_id) = (
    SELECT COUNT(product_id)
    FROM Products
    WHERE category_name = 'Electronics'
);
```

3. Retrieve the highest earning employee per department only if their salary is above the company's average.

Explanation: We use the **RANK()** window function to identify the top earner in each department. We then join this result with a subquery that calculates the company's overall average salary and filter based on both conditions.

Query:

```
SELECT dept_rank.department_id, dept_rank.name, dept_rank.salary
FROM (
    SELECT
        *,
        RANK() OVER (PARTITION BY department_id ORDER BY salary DESC) as
    rnk
    FROM Employees
) AS dept_rank
WHERE dept_rank.rnk = 1 -- Condition 1: Highest earner in department
AND dept_rank.salary > (SELECT AVG(salary) FROM Employees); -- Condition
2: Above company average
```

4. Find all employees who worked on all projects assigned to their department.

Explanation: This is another "Division" problem. We compare the count of unique projects an employee has worked on (within their department) against the total count of projects assigned to that department.

Query:

```
SELECT e.employee_id, e.name
FROM Employees e
JOIN EmployeeProjects ep ON e.employee_id = ep.employee_id
JOIN Projects p ON ep.project_id = p.project_id AND e.department_id =
p.department_id -- Ensure project belongs to employee's department
GROUP BY e.employee_id, e.department_id
HAVING COUNT(DISTINCT ep.project_id) = (
    SELECT COUNT(project_id)
    FROM Projects
    WHERE department_id = e.department_id
);
```

5. List employees whose salary increased more than 3 times in the last year (from salary_history).

Explanation: We filter the SalaryHistory table for records within the last year, group by employee ID, and count the number of increases where the new salary is strictly greater than the old salary.

Query:

```
SELECT employee_id
FROM SalaryHistory
```

```
WHERE change_date >= DATE_SUB(CURDATE(), INTERVAL 1 YEAR) -- Filter
records in the last year
GROUP BY employee_id
HAVING SUM(CASE WHEN new_salary > old_salary THEN 1 ELSE 0 END) > 3; --
Count increases
```

6. Get month-wise user retention (users who logged in this month AND last month).

Explanation: For a given month, we find the set of users who logged in during that month and the set who logged in during the previous month. We then use **INTERSECT** to find the common users (retained users) between the two months.

Query (Finding retention for the current month vs. last month):

```
SELECT
    COUNT(DISTINCT t1.user_id) AS retained_users_count
FROM UserLogins t1
INNER JOIN UserLogins t2
    ON t1.user_id = t2.user_id
WHERE DATE_FORMAT(t1.login_date, '%Y-%m') = DATE_FORMAT(CURDATE(), '%Y-
%m') -- Logged in this month
AND DATE_FORMAT(t2.login_date, '%Y-%m') = DATE_FORMAT(DATE_SUB(CURDATE(),
INTERVAL 1 MONTH), '%Y-%m'); -- Logged in last month
```

7. Get customers who placed an order in at least 10 different months of the year.

Explanation: We group orders by customer_id and use **COUNT(DISTINCT MONTH())** to count the number of unique months in which they placed an order, filtering those who meet the minimum count (10).

Query:

```
SELECT customer_id
FROM Orders
WHERE YEAR(order_date) = YEAR(CURDATE()) -- Filter for current year
GROUP BY customer_id
HAVING COUNT(DISTINCT MONTH(order_date)) >= 10;
```

8. Find employees who changed departments more than once (using history table).

Explanation: We group the DepartmentHistory records by employee ID and count the number of distinct departments they worked in. We filter those where the count is greater than two

(the original department plus one change equals two departments; changing *more* than once means at least three distinct departments).

Query:

```
SELECT employee_id
FROM DepartmentHistory
GROUP BY employee_id
HAVING COUNT(DISTINCT department_id) > 2; -- Switched more than once (at
least 3 departments)
```

JAVA Interview Buddy